

底层的模块，然后利用底层模块提供的功能与服务进一步构建上层的模块。例如，第 1 章介绍操作系统的简明结构时，也是通过将操作系统分为内核、系统服务、应用框架与应用程序等层次来控制操作系统的复杂性。

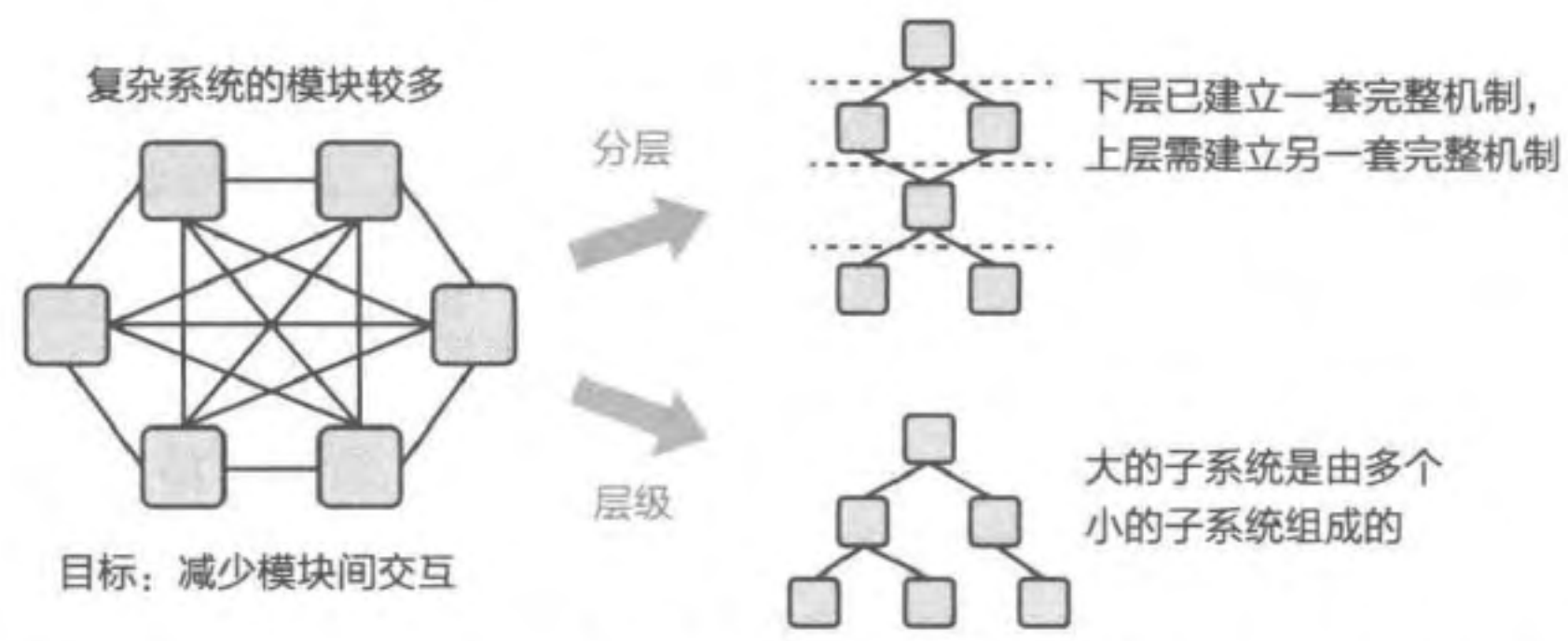


图 2.1 过多的模块会导致交互非常复杂，需要通过分层与层级来降低复杂性

小知识：分层

“计算机科学中的所有问题都可以通过另外一个分层来解决，除了太多间接分层的问题外。”（All problems in computer science can be solved by another level of indirection, except for the problem of too many layers of indirection.）

——Butler Lampson 1992 年图灵奖获奖报告（引自 David Wheeler）

对于一个已经构建好的复杂系统，分层也是一个很好的解决问题的方式。例如，针对数据中心的服务器性能越来越强大但利用率低下且管理困难等问题，系统虚拟化通过在操作系统之下再构建一层虚拟机监控器层，有效地提升了资源利用率，提高了性能与错误隔离的能力，并且降低了管理难度。因而，系统虚拟化也成为云计算的关键支撑技术之一。

层级。层级是另外一种模块的组织方式：首先将一些功能相近的模块组成一个具有清晰接口的自包含子系统，然后再将这些子系统递归式地组成一个具有清晰接口的更大的子系统。层级是日常生活中常见的组织形式。例如，在公司的组织架构中，一个经理管理一组成员，一组经理构成一个部门，多个部门构成一个事业部，多个事业部构成一个公司。在操作系统中，虚拟内存是一个子模块，与物理内存分配、缺页异常处理、页换入换出等一同构成内存管理模块，内存管理模块再与进程管理模块、设备驱动模块等一起构成操作系统内核。

分层与层级

分层和层级这两个概念有点像，有些时候不容易区分。简要而言，分层是指不同类模块之间的层次化，而层级则是指同类模块之间的分层。两种方法可以组合起来一

起降低模块的复杂性。例如，操作系统包括内核层、系统服务层与应用框架层等，而操作系统内核中则可以通过如上的层级方式进行组织。

M.A.L.H 是操作系统中降低复杂性与组织各种功能模块的有效方法。早期的操作系统（如 MS-DOS）缺乏模块化能力，不区分用户程序和操作系统的运行环境，使得一个应用的错误就可能导致整个系统崩溃。后来，操作系统设计人员利用处理器提供的特权级^①对系统进行分层以避免错误的传播，并进一步针对各种不同的需求设计了各种不同的操作系统架构。

2.3 操作系统内核架构

本节主要知识点

- 操作系统的内核有哪些常见的架构？
- MS-DOS 的架构有什么优点和缺点？
- 一个应用能否运行在多个内核之上？
- 微内核与宏内核到底孰优孰劣？

随着操作系统功能的不断增多和代码规模的不断扩大，提供合理的层级结构，对于降低操作系统的复杂性、提升操作系统的安全性与可靠性来说变得尤为重要。图 2.2 列举了一些常见的操作系统内核架构，下面我们对这些架构进行简要的介绍和分析。

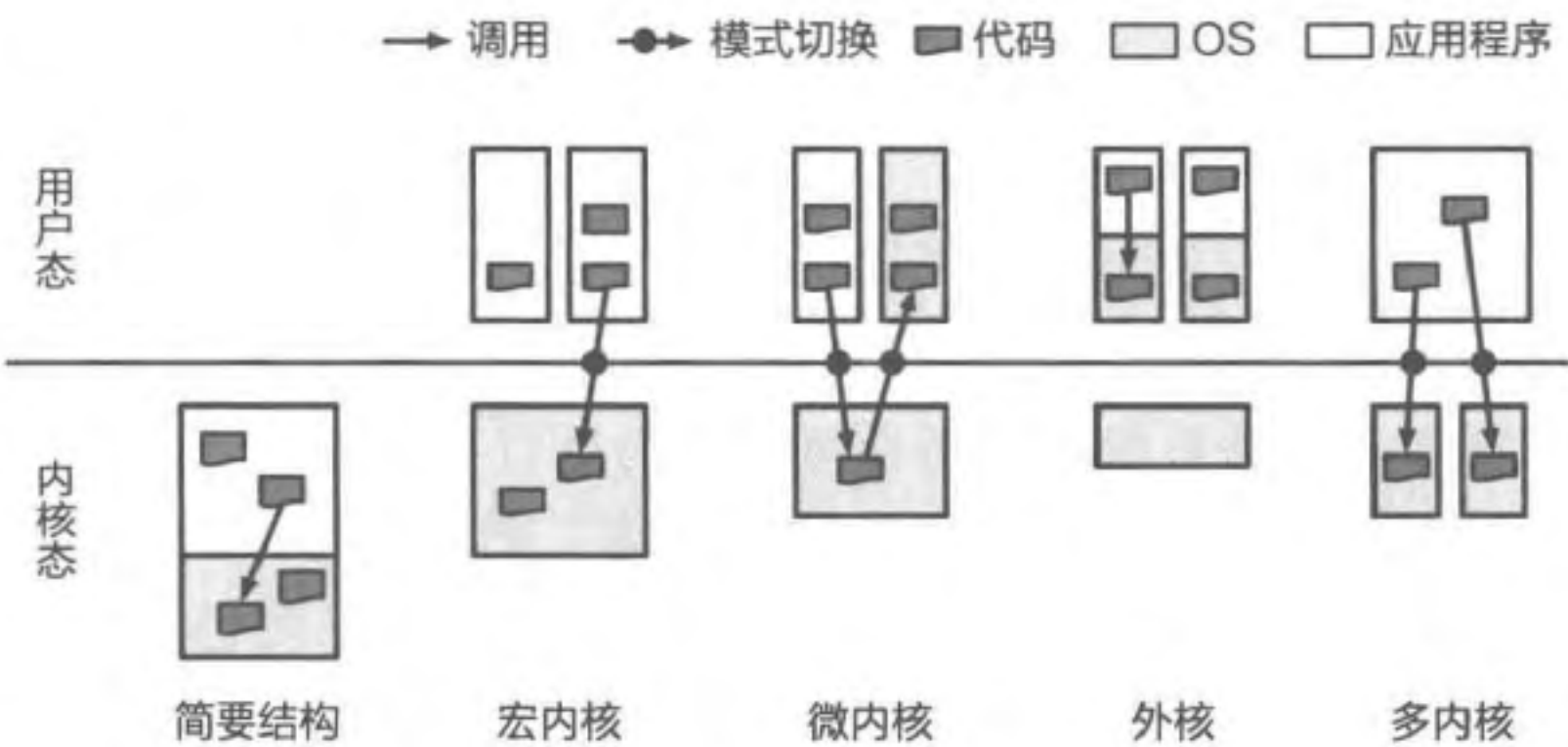


图 2.2 操作系统内核架构的频谱：简要结构（如 MS-DOS）、宏内核（如 UNIX/Linux）、微内核、外核与多内核等

① 处理器特权级指用户态与内核态，后文会深入介绍。

2.3.1 简要结构

一些功能较为简单的操作系统会选择将应用程序与操作系统放置在同一个地址空间 (Address Space) 中, 以同样的权限运行, 无须底层硬件提供复杂的内存管理、特权级隔离等功能。MS-DOS (Microsoft Disk Operating System) 是采用简要结构的一个典型例子。该结构的一个优势在于, 应用程序对操作系统服务的调用无须切换地址空间和权限层级, 因此更为高效^①。但缺点也同样明显: 缺乏良好的隔离能力, 任何一个应用或操作系统模块出现问题, 均有可能使整个系统崩溃。随着操作系统功能的不断增加, 简要结构会使操作系统的设计与实现难度越来越高, 难以持续演进。

尽管缺乏隔离能力, 但简要结构的操作系统依然采用了一定的模块化与层次结构以降低复杂性。图 2.3 展示了 MS-DOS 的内部结构: MSDOS.Sys 模块通过命令行接口与用户交互, 并负责与设备驱动交互以实现对硬件设备的管理; I/O 子系统 (IO.Sys) 实现对硬件设备 I/O 访问的管理, 并以 I/O 请求作为抽象, 为 MSDOS.Sys 和驱动程序 I/O 提供服务。

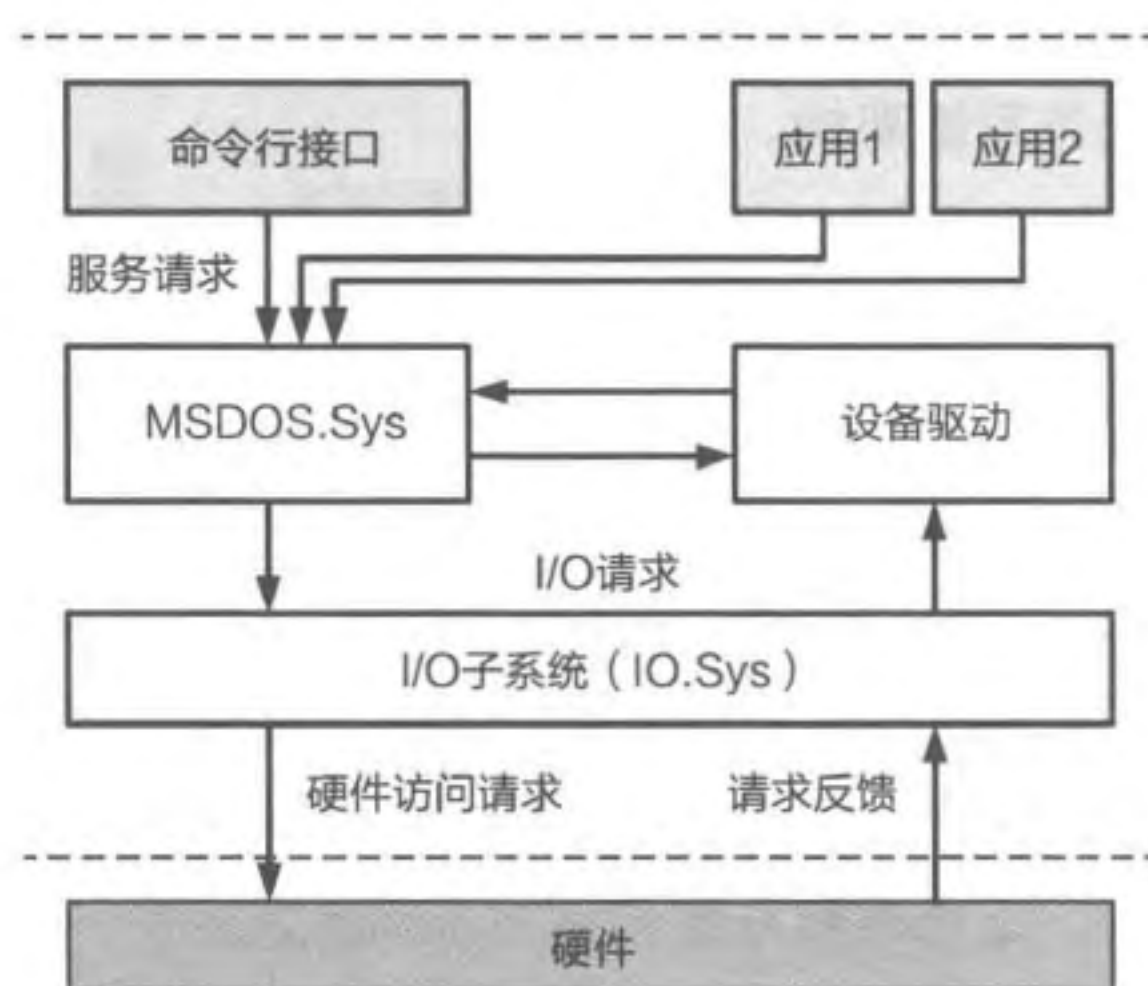


图 2.3 MS-DOS 的内部结构

除了 MS-DOS 外, 当前一些采用简要结构的操作系统还包括 FreeRTOS^②与 $\mu\text{C}/\text{OS}$ ^③等。这些操作系统主要运行在微控制单元 (MicroController Unit, MCU) 等相对比较简单硬件上, 这些硬件通常没有提供现代意义上的内存管理单元 (Memory Management Unit, MMU), 隔离能力较弱或缺失, 难以运行 (往往也不需要运行) 复杂的操作系统。

2.3.2 宏内核

宏内核 (Monolithic Kernel) 又称单内核, 其特征是操作系统内核的所有模块 (包括进程调度、内存管理、文件系统、设备驱动等) 均运行在内核态, 具备直接操作硬件的能力, 这类操作系统包括 UNIX/Linux、FreeBSD 等。图 2.4 展示了典型的宏内核架构。

由于操作系统内核的功能日趋复杂, 宏内核架构的操作系统也逐步采用 M.A.L.H 方法来控制其不断增加的复杂性。下面是一些典型的方法。

模块化。现代操作系统内核如 UNIX、Linux、Windows 等均采用模块化的策略来组

① 在 MS-DOS 中, 应用程序调用系统服务时依然需要走异常处理流程, 而不是函数调用。

② 参见 <https://www.freertos.org/>。

③ 参见 <https://www.micrium.com/rtos/kernels/>。

织各个功能。在操作系统代码中，通常会有类似 `arch/arm/` 的目录，封装与体系结构相关的功能实现。为进一步提高功能的可扩展性，现代操作系统通常还提供可加载内核模块（Loadable Kernel Module, LKM）机制。例如，当前大部分设备驱动是以可加载模块的形式存在的，与内核的其他模块解耦，使驱动开发与驱动加载更加方便、灵活。近年来，Linux 的 eBPF 机制发展迅速，它通过在内核中构建一个虚拟机，动态加载 eBPF 程序在内核中执行，比可加载内核模块更安全。

抽象。现代操作系统内核均广泛采用抽象来降低复杂性并提高可维护性。例如，UNIX 将文件作为一个重要的抽象，提出“一切皆文件”（Everything is a file），将数据、设备、内核对象等均抽象为文件，并为上层应用提供统一的接口。

小知识：Linux 的 eBPF 机制

Linux 的 eBPF 虚拟机不同于 VMware、KVM 所提供的系统虚拟机。eBPF 使用自定义的 64 位 RISC 指令集，且对所执行的 eBPF 程序有严格的限制。比如，所有内存访问都需要类型检查，仅支持有界循环，程序的指令数有上限，等等。其目的是验证 eBPF 程序不会破坏内核的安全。

分层。宏内核架构的操作系统一开始就采用了分层的架构。例如，图灵奖获得者 Edsger Dijkstra 在 1968 年提出的“THE”操作系统^[4]将操作系统分成 6 层，如图 2.5 所示。现代操作系统内核也均存在一定程度的分层结构，以更好地组织各种功能。图 2.6 展示了 Linux 文件系统的分层结构。

层级。层级的概念同样被广泛应用于内核的资源管理中，如调度子系统中对进程优先级的分类，控制组（Cgroups）对进程层级的分类，内存分配器对不同内存的分类等。

通过各种复杂性控制方法，Linux 已经演进为一个超过 2 800 万行代码的复杂系统，成为世界上最大的开源协作项目，2021 年有超过 5 000 名开发者为 Linux 提交补丁来修复问题以及添加新功能。然而 Linux 同样面临挑战：通用的、适用于大部分场景的设计，常常意味着很难满足特定场景下对安全性、可靠性、实时性等方面的需求；同时，在一个庞大的系统中进行创新也变得越来越困难，这使得一些较大的创新（如网络、文件系统、设备驱动等）开始向用户态迁移。

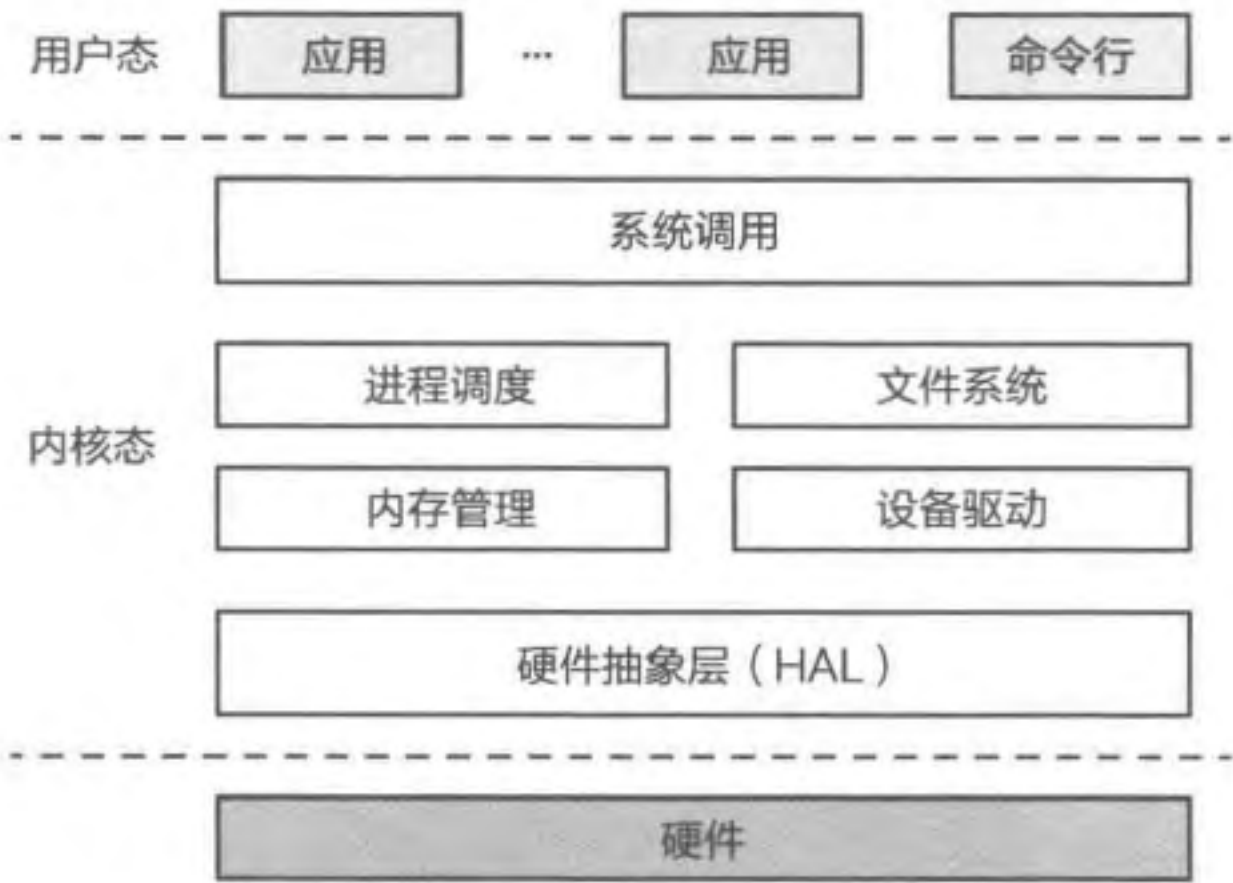


图 2.4 宏内核的基本结构



图 2.5 “THE” 操作系统的分层结构

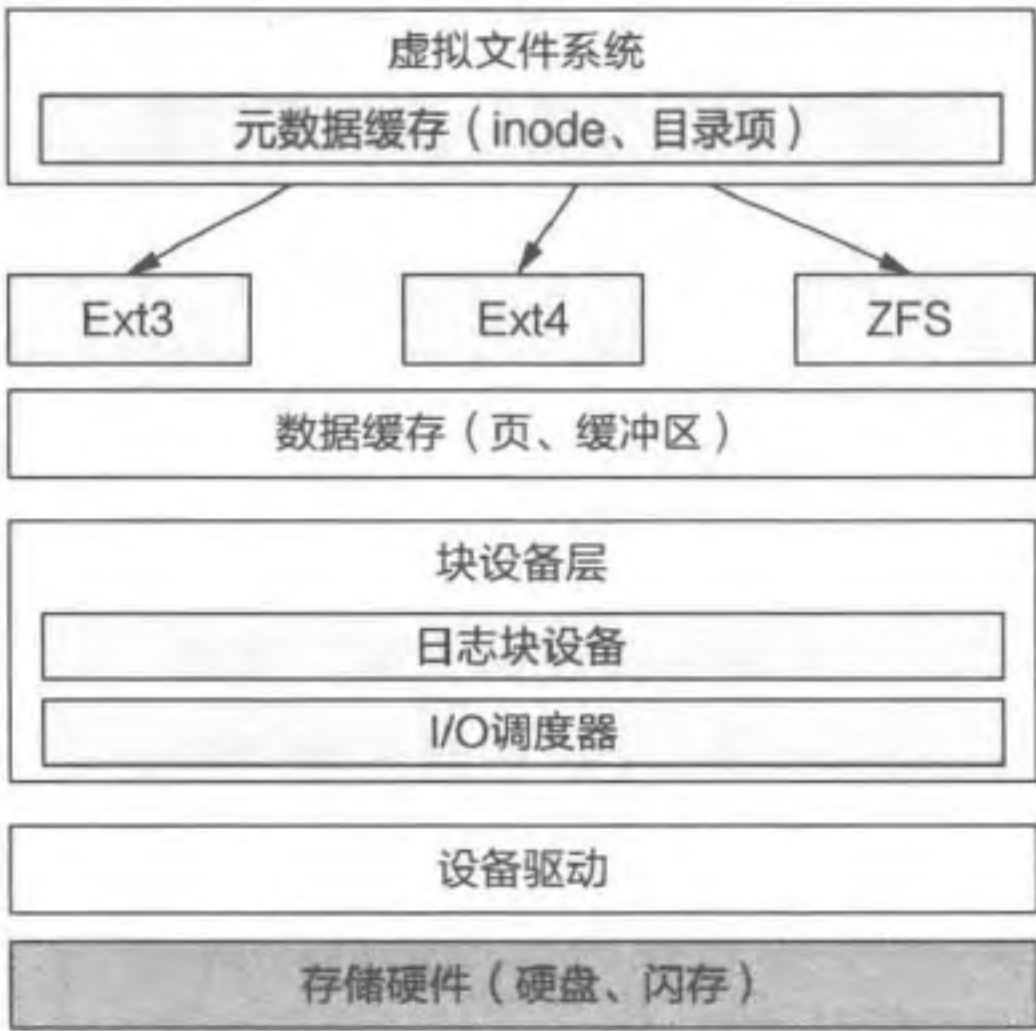


图 2.6 Linux 文件系统的分层结构

2.3.3 微内核

随着宏内核操作系统的内核功能不断增长，系统的复杂性也持续增加，在可靠性、安全性等方面导致了更多的问题。这是因为在宏内核架构下，所有内核模块均运行在特权级，一个单点的错误就可能对整个系统崩溃或者被攻破。而系统很难避免 Bug，哪怕系统出自具有极强编程能力的操作系统内核程序员之手。Andrew Tanenbaum 等人的论文^[5]提到，一般的工业界系统中每千行代码会有 6 ~ 16 个缺陷。虽然很多缺陷在正常运行时不会被触发，部分缺陷即使触发也不会引起严重后果，但对于一个千万行代码级的软件而言，潜在的重大缺陷数量也是触目惊心的。

因此，研究人员尝试对宏内核架构的操作系统进行解耦，将单个功能或模块（如文件系统、设备驱动等）从内核中拆分出来，作为一个独立的服务（Service）部署到独立的非特权运行环境中；内核仅保留极少的功能，为这些服务提供通信等基础能力，使其能够互相协作以完成操作系统必需的功能。这种架构被称为微内核（Microkernel）。在微内核架构下，服务与服务之间是完全隔离的，单个服务即使出现故障或受到攻击，也不会直接导致整个操作系统崩溃或被攻破，从而能有效提高操作系统的可靠性与安全性。此外，微内核架构实现了机制与策略的进一步分离，可更方便地为不同场景定制不同的服务，从而更好地适应不同的应用需求。

小知识：最早的微内核操作系统

一些读者可能认为微内核架构是一种比较新的设计。事实上，早在 1969 年 UNIX 系统开始设计的时候，类似微内核架构的操作系统就已经出现。Per Brinch Hansen 开发的 RC 4000 多路编程系统在历史上第一次将操作系统分离为各个交互的功

能组件，以及一个负责消息通信的内核^[2]。Per Brinch Hansen 在 RC 4000 中首次提出分离机制与策略的原则，并且提出了管程（Monitor）的概念^①。

微内核的发展到目前为止经历了三代。Mach 是第一代微内核的代表。1975 年，Mach 起源于罗切斯特大学，后来主要在卡内基·梅隆大学开发^②。Mach 将很多内核功能以单独服务的形式运行在用户态，然而，Mach 对于进程间通信（Inter-Process Communication, IPC）的设计过于通用，加上 Mach 微内核自身资源占用过大（包括内存与 CPU 缓存等）的问题，使得其性能与同时期的宏内核相比存在差距^[6]，甚至有人据此将微内核与性能差关联起来。

微内核的性能一定差吗？德国国家信息技术研究中心的 Jochen Liedtke 深入分析了 Mach 微内核系统的性能，指出较差的性能不是微内核的必然结果。Jochen 认为，高性能 IPC 的设计与实现必然是与体系结构相关的：过度抽象将极大影响 IPC 的性能，而利用体系结构的特性进行优化则可将 IPC 的性能提升到极致^[7]。为此，Jochen Liedtke 设计并实现了 L4 微内核系统，并提出了微内核的最小化原则：应尽可能将操作系统的功能放置在内核态以外，只有在将其放在内核态以外会影响整个系统的功能时，才能被放置在内核态。通过高性能的 IPC 实现以及极小化的微核（即微内核系统的内核态部分，又称 μ kernel），微内核架构操作系统的性能可以达到甚至超过同时期的宏内核架构操作系统^[8]。L4 被认为是第二代微内核操作系统的代表。

随着 L4 等微内核操作系统在实时、高安全等场景的广泛应用，研究人员开始进一步增强微内核的安全性。EROS^[9] 首次将 Capability（能力）机制引入微内核操作系统中，并高效地实现了该机制（详见 21.2.6 节）。Capability 机制允许更精确、更细粒度地授予不同应用程序对内核对象的调用权，从而更好地提升系统的安全性。seL4^[10-11] 是一个典型的基于 Capability 机制的微内核，谷歌正在实现的 Fuchsia 微内核操作系统同样基于 Capability 机制实现了访问控制。此外，seL4 还引入了形式化证明方法（详情见第 23 章），通过数学方式证明了其微核部分满足从设计到实现的一致性，以及微核上的服务满足互不干扰（Non-interference）等属性。这些安全增强能力成为第三代微内核架构操作系统的重要特征。

宏内核与微内核

自宏内核与微内核两种操作系统架构出现以来，人们对两者的优劣势与特点展开了多次深入的讨论。当前，随着一些新场景、新诉求的出现，类似微内核架构的操作

① 管程：将进程同步与面向对象编程有效结合起来的一种抽象。

② 1979 年，Mach 的主要设计者 Richard Rashid 从罗切斯特大学离开并加入卡内基·梅隆大学。有意思的是，Richard Rashid 也是微软研究院的创始人，Mach 的设计深刻影响了 Windows NT 操作系统的设计。

系统架构再次受到关注。

- 弹性扩展能力：对于宏内核来说，很难仅仅通过简单的裁剪或扩展，使其支持资源诉求从 KB 到 TB 级别的场景。
- 硬件异构性：异构硬件往往需要一些定制化的方式来解决特定问题，这种定制化对于宏内核来说很难得到长期的支持。
- 功能安全：由于宏内核在故障隔离和时延控制等方面的缺陷，截至目前尚无通过高等级功能安全认证（例如，汽车行业的 ASIL-D）的先例。
- 信息安全：宏内核架构的操作系统存在较大的信息安全隐患，例如内核态驱动容易导致低质量的驱动代码入侵内核，粗粒度权限管理容易带来权限漏洞等。
- 确定性时延：由于宏内核架构的资源隔离较为困难，且各模块的耦合度高，导致难以控制系统调用的时延，因此较难做到确定性时延。即便为时延做一些特定优化（例如 Linux-RT 补丁^①），时延抖动仍然较大。

在真实世界中，正如体系结构领域的 RISC 与 CISC 之争一样，宏内核与微内核往往也会互相借鉴。例如，Intel 处理器采用了 CISC 的指令集架构，但其微架构实现则采用了 RISC；类似地，Linux 等宏内核架构操作系统也采用了一些微内核的设计思想。再如，尽管 Linux 的创始人 Linus Torvalds 在 20 世纪 90 年代与 Andrew Tanenbaum 论战^②时表明将驱动放到用户态是个不靠谱的想法，但近期 Linux 也逐步采用了一些用户态驱动框架（如 UIO 与 VFIO 等）；Android 操作系统在 Treble 项目中同样将部分驱动放到了用户态，并通过名为 Binder 的 IPC 机制来与这些驱动进行交互。

小思考

小的操作系统内核就是微内核吗？

不是。有一些操作系统内核（如 FreeRTOS、 μ C/OS-II 等）虽然很小，但是不具备现代意义上操作系统的功能，包括虚拟内存、用户态和内核态分离等。因此它们应该被归为本章提到的简要结构内核。

2.3.4 外核

操作系统内核在硬件管理方面的两个主要功能是资源抽象与多路复用（Multiplexing）。其中，对硬件资源的抽象存在两方面的问题：

① 参见 https://rt.wiki.kernel.org/index.php/Main_Page。

② 参见 https://en.wikipedia.org/wiki/Tanenbaum%E2%80%93Torvalds_debate。

- 过度的硬件资源抽象可能会带来较大的性能损失，违反“抽象但不隐藏能力”（Abstract but don't hide power）原则^[12]。
- 操作系统所提供的硬件资源抽象是针对所有应用的通用抽象，这些抽象对一些具体的应用（如数据库、Web 服务器等）来说往往不是最优的选择。

为此，MIT 的 Dawson Engler 和 Frans Kaashoek 等研究者于 1995 年提出了外核（Exokernel）架构^[13]。他们观察到，在许多场景中，应用比操作系统更了解该如何抽象与使用硬件资源，因此，应当由应用来尽可能地控制对硬件资源的抽象。为了降低应用开发的复杂度，他们同时提出了库操作系统（LibOS）的概念，将对硬件的抽象封装到 LibOS 中，并与应用直接链接；开发者可选择已有的 LibOS，或选择自己开发 LibOS。操作系统内核则只负责实现硬件资源在多个 LibOS 之间的多路复用，并管理这些 LibOS 实例的生命周期。通过这种解耦，外核架构将对硬件的抽象从原本不可选择的机制变成可选择的策略。

图 2.7 展示了外核操作系统的架构。外核架构可为不同的应用提供定制化的高效资源管理：按照不同应用领域的要求，将对硬件资源的抽象模块化为一系列的库（即 LibOS）。这样的设计带来两个主要好处：

- 可按照应用领域的特点与需求，动态组装成最适合该应用领域的 LibOS，最小化非必要的代码，从而获得更高的性能。
- 拥有特权级的操作系统内核可以做到非常小，并且由于多个 LibOS 之间的强隔离，可以提升整个计算机系统的安全性及可靠性。

当前有很多领域已经使用了外核架构，例如在一些功能受限、对操作系统接口要求不高但对性能和时延特别敏感的嵌入式场景中，通过 LibOS 来运行应用业务，从而将数据面（Data-plane，负责数据的处理与转发等）与控制面（Control-plane，负责设备的配置与管理等）分离。此外，当前云计算平台的很多容器（Container）架构采用了脱胎于外核架构的 Unikernel（本质上是一个 LibOS），通过将虚拟机监控器作为支撑 Unikernel/LibOS 运行的内核，从而支持对高性能业务的独立部署。

然而，外核架构的劣势在于，LibOS 通常是为某种应用定制的，缺乏跨场景的通用性，应用生态差。因此，外核架构较难用于功能要求复杂、生态与接口丰富的场景，因为这意味着要将 LibOS 做得过于复杂，甚至相当于一个完整的宏内核，从而丧失了外核架构带来的性能、安全等优势。此外，不同的 LibOS 通常会实现相同或类似的功能，容易造成代码冗余，因此对于资源受限的场景，通常需要一些跨地址空间的代码去重（Code Deduplication）或内存共享机制来减少内存开销。

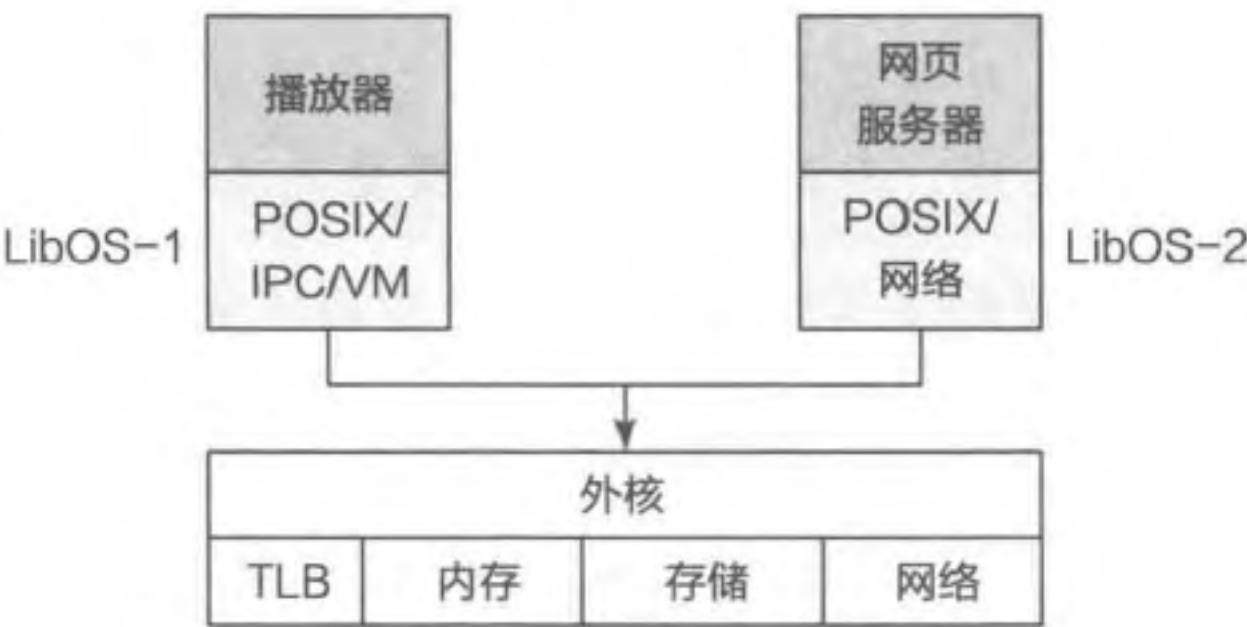


图 2.7 外核操作系统的架构

外核与微内核

由于外核架构下运行在特权级的操作系统内核的功能与微内核类似，可以做到非常小，一些读者可能容易将外核架构与微内核架构混淆。然而，两者是具有明显区别的：

- 外核架构将多个硬件资源切分为一个个切片，每个切片中保护的多个硬件资源由 LibOS 管理并直接服务一个应用^①；而微内核架构则是让一个操作系统模块独立运行在一个地址空间来管理一个具体的硬件资源，为操作系统中的所有应用提供服务。
- 外核架构中，运行在特权级的内核主要为 LibOS 提供硬件的多路复用能力并管理 LibOS；而微内核架构中，内核主要提供进程间通信（IPC）功能。
- 外核架构在面对功能与生态受限场景时可通过定制化 LibOS 获得非常高的性能，而微内核架构则需要更复杂的优化才能获得与之类似的性能。

2.3.5 其他操作系统内核架构

多内核（Multikernel）架构。当前的硬件结构呈现两个主要特性：首先，多核乃至众核^②架构的流行使得一个服务器中通常存在成百上千个处理器核；其次，Dennard 缩微定律^③的终结以及应用需求的多样化使得处理器走向异构化，甚至是动态异构化，也就是一个处理器上可能集成多种功能、性能甚至指令集结构各异的处理器核。这对操作系统内核的架构如何管理异构众核提出了新的挑战。

多内核是苏黎世联邦理工学院、微软研究院、雷恩高等师范学校等联合研制的一种实验型的操作系统内核架构^[14]。基于硬件的众核与异构特性，多内核的想法是将一个众核系统看成由多个独立处理器核通过网络互联而成的分布式系统。与传统的操作系统类似，多内核架构仍然假设硬件处理器提供全局共享内存的语义，但对于不同处理器核之间的交互，它提供了一层基于进程间通信的抽象，从而避免了处理器核之间通过共享内存进行隐式共享。如图 2.8 所示，多内核架构在每个 CPU 核上运行一个独立的操作系统节点，节点间的交互由操作系统节点之上的进程间通信来完成。通过这种架构，多内核可以避免传统操作系统架构中复杂的隐式共享带来的性能可扩展性瓶颈，并且由于不同处理器核上运行的操作系统节点是独立的而且可以不同，从而非常容易支持异构处理器架构。多内核架构的不足之处在于：不同节点之间的状态存在冗余，导致一定的资源开

① 在这里应用是指用于完成某个功能的应用集合，可以由多个小的应用构成。

② 一般认为 8 个及以下处理器核为多核，多于 8 个处理器核称为众核。

③ 由 DRAM 的发明者、IBM Fellow Robert H. Dennard 在 1974 年提出，主要观点是随着晶体管变得越来越小，它们的功率密度保持不变，从而使功率使用与面积成比例。

销；上层应用必须使用多内核提供的进程间通信接口才能进行交互，需要移植现有应用才能适应多内核架构；绝对性能方面并不一定存在优势。

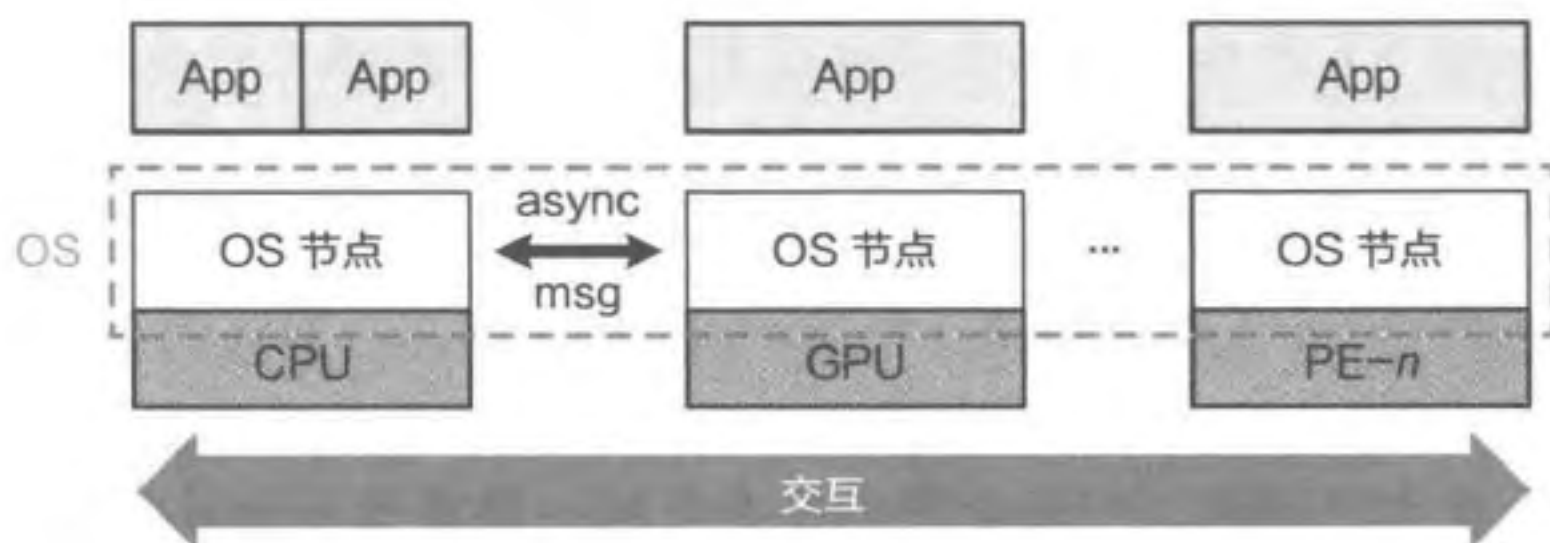


图 2.8 多内核操作系统的架构

混合内核架构。由于设计需求的多样化，现实中的操作系统往往融合了多种架构的设计思想。图 2.9 展示了当前微内核、宏内核、外核、多内核等架构的演进及其在现实操作系统中的体现。例如，苹果操作系统的内核 XNU 是 Mach 微内核与 BSD UNIX 的混合体；Windows NT 操作系统的内核也采用了微内核设计思想，但为了实现更低的操作时延，选择将一部分系统服务（特别是与图形界面相关的服务）运行在内核态；Linux 虽然是宏内核架构，但近期也开始支持用户态的驱动框架（如 UIO^①与 VFIO^②等）。

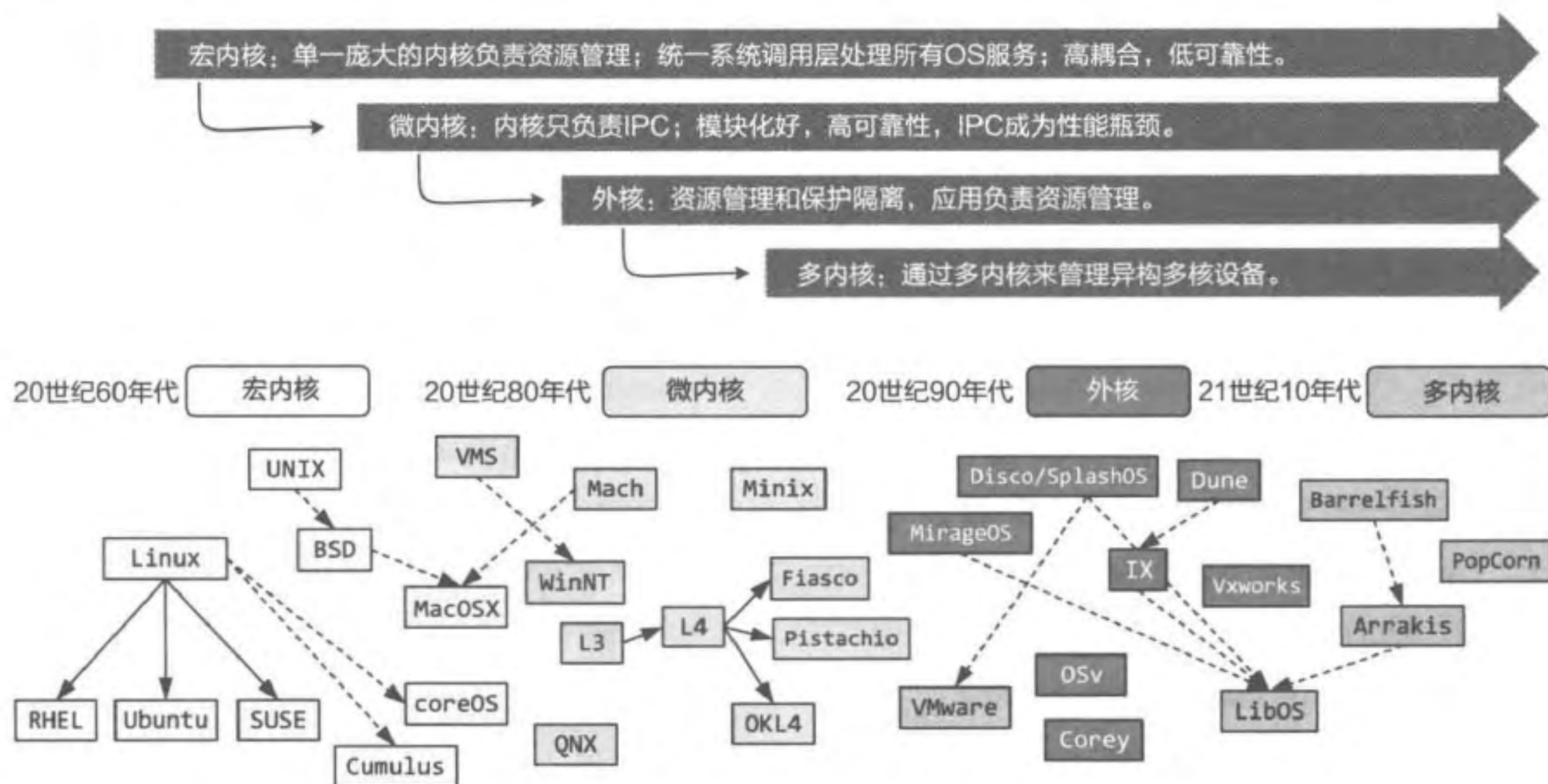


图 2.9 操作系统的架构组合及其演进

① 参见 <https://www.kernel.org/doc/html/v4.18/driver-api/uio-howto.html>。

② 参见 <https://www.kernel.org/doc/html/latest/driver-api/vfio.html>。

小知识：Windows 也采用了微内核的设计思想吗？

是的。事实上，微软研究院的创始人 Richard Rashid 就是第一代微内核 Mach 的设计者。从 Windows NT 开始，微软将与处理器和体系结构相关的功能放入一个很小的微内核，而将其他功能（如进程管理器、虚拟内存管理器、进程间通信管理器、I/O 管理等）以模块化的方式实现，称为 executive。与纯微内核不同的是，Windows NT 将微内核与 executive 链接到一个模块（称为 ntoskrnl.exe），共同运行在处理器的特权模式下。这意味着不同 executive 的函数间可以互相直接调用，具有很低的时延，但也意味着一旦某个函数出现问题，可能会影响整个系统，隔离性不如纯微内核。

2.4 操作系统框架结构

本节主要知识点

- ❑ 为什么说 Android 的应用框架与微内核架构相似？
- ❑ Android 成为最流行的移动终端操作系统，这与它的结构有哪些关系？

如 1.2 节所述，现代操作系统一般由操作系统内核和操作系统框架（包含系统服务与应用框架）构成，本节将简要介绍两个操作系统（Android 和 ROS）的框架。为了简化开发并提供可演进性与可维护性等，许多操作系统框架的结构设计借鉴了类似微内核架构的思想，将操作系统系统框架进行组件化与服务化，并提供进程间通信以支撑各个组件进行交互。

2.4.1 Android 系统框架

Android 是当前最流行的移动终端操作系统。截至 2019 年年底，Android 已经被部署到全球 74.13% 的智能终端设备上。Android 的设计目标之一是方便各个厂商适配，整体采用了更加商业友好的 Apache Software License。与 GPL（GNU Public License）不同的是，Apache Software License 不要求使用并修改源码的开发者重新开放源码，而只需要在每个修改的文件中保留 License 并表明所修改的部分。



图 2.10 展示了 Android 操作系统的

图 2.10 Android 操作系统的架构